

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du cours. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Ne pas oublier d'indiquer ses noms et prénoms.

1. Répondez aux questions suivantes en justifiant brièvement la réponse.

- **Soit $\mathcal{L}$ un langage rationnel, peut-il être aussi algébrique ?**

Oui. L'ensemble des langages rationnels est un sous-ensemble de celui des langages algébriques. Autrement dit, tous les langages rationnels sont aussi algébriques.

- **Soit $\mathcal{L}$ un langage algébrique, un sur-ensemble de $\mathcal{L}$ peut-il être rationnel ?**

Oui. Par exemple, Σ^* est un sur-ensemble de tous les langages, en particulier des langages algébriques.

- **Soit $\mathcal{L}$ un langage algébrique, peut-il ne pas être rationnel ?**

Oui. On a vu le langage $\{ a^n b^n \mid n \geq 0 \}$ qui n'est pas rationnel, mais qui est algébrique.

Ce sont des questions largement rebattues en cours et très proches de questions posées au premier CC et dont les réponses attendues ont été publiées. Conclusion : aucune excuse pour ne pas répondre correctement.

2. Quelle est le principal ajout conceptuel des grammaires par rapport aux expressions régulières ? Développez un peu.

Le principal ajout conceptuel est celui du vocabulaire non-terminal, c'est-à-dire le vocabulaire des structures syntaxiques. Ce vocabulaire est couramment noté Σ_{NT} . Le vocabulaire des symboles qui servent à former les mots d'un langage est appelé vocabulaire terminal, noté Σ_T .

Là encore, c'est une question très rebattue.

3. Qu'est-ce qui caractérise les grammaires algébriques ?

Les règles des grammaires algébriques ont une tête formée d'un unique symbole non-terminal et un corps constitué d'une séquence quelconque, possiblement vide, de symboles terminaux et non-terminaux.

C'est une définition, à connaître absolument.

4. Rappelez le théorème qui est similaire à celui de Kleene pour les langages algébriques. Bonus pour une explication de pourquoi ces théorèmes sont importants.

Théorème : un langage est algébrique (c-à-d. engendré par une grammaire algébrique) ssi il est reconnu par un automate à pile (c-à-d. tous ses mots et seulement eux sont acceptés par l'automate à pile).

Bonus : ce théorème et celui de Kleene sont importants car ils mettent en relation des moyens commodes de décrire des langages (grammaires algébriques et expressions régulières) et des moyens commodes de vérifier si un mot appartient à un langage (automates à pile et automates finis). Sans eux, on n'aurait pas la possibilité d'automatiser la production des automates.

Vu en cours, et même une variante a été posée au 1^{er} CC.

5. Qu'est-ce qu'une grammaire algébrique étendue ? Bonus pour une explication de leurs avantages.

Une grammaire algébrique étendue est une grammaire algébrique où le corps des règles est une expression régulière formée sur le vocabulaire terminal et le vocabulaire non-terminal.

Cette forme permet d'exprimer de façon plus concise des traits de langage très courants comme les séquences de trucs, ou les séquences de trucs séparés par des machines. Cette forme évite aussi assez facilement des constructions de grammaire qui posent problème à certains analyseurs syntaxiques : ex. les récursivités à gauche.

Vu en cours, TD et TP.

6. Qu'est-ce que l'ambiguïté ? Pourquoi est-ce grave ?

Une grammaire est ambiguë si plusieurs arbres de dérivation différents peuvent avoir le même mot des feuilles.

C'est grave car (1) l'arbre de dérivation révèle la structure interne des mots, et (2) la structure interne des mots indique leur sémantique. Si un mot a plusieurs structures internes possibles, il risque d'avoir plusieurs sémantiques, et c'est quelque chose qu'on ne peut presque jamais se permettre dans un traitement automatique.

Vu en cours et TD.

Soit l'alphabet $\Sigma = \{ N, E, S, W, 0, 1 \}$ où N, E, S, W se lisent Nord, Est, Sud et Ouest.

Soit $\mathcal{L}_{\text{step}}$ le langage des notations d'étapes de la forme « direction suivie de nombre non nul de pas en binaire » (on négligera la question des 0 en tête) : ex. W011.

Soit $\mathcal{L}_{\text{journey}}$ le langage des notations d'itinéraires de la forme « suite d'étapes » : ex. W011S10E01.

On souhaite composer une grammaire algébrique qui décrit le langage $\mathcal{L}_{\text{journey}}$.

7. La grammaire suivante sensée décrire le langage $\mathcal{L}_{\text{journey}}$ présente un certain nombre d'anomalies. Listez-les.

- | | |
|--|--|
| 1. $\text{Direction} \rightarrow ('N' \mid 'E' \mid 'S' \mid 'W')$ | 5. $\text{Step} \rightarrow \text{Direction Bins}$ |
| 2. $\text{Bin} \rightarrow ('0' \mid '1')$ | 6. $\text{Journey} \rightarrow \text{Step}$ |
| 3. $\text{Bins} \rightarrow \text{Bins Bin}$ | 7. $\text{Journey} \rightarrow \text{Journey Journey}$ |
| 4. $\text{Bins} \rightarrow \varepsilon$ | |

Règle 4 : Bins peut engendrer une liste vide de bits ce qui contredit la spécification.

Règle 3 : Cette règle est récursive à gauche. Ce n'est pas forcément une erreur, mais ça demande d'y réfléchir selon l'analyseur syntaxique qui va être employé. Rappel : LL(x) est allergique à la récursivité gauche.

Règle 7 : Cette règle est ambiguë. Par exemple, S1W1S1 peut être analysé comme S1 suivi de W1S1, ou bien comme S1W1 suivi de S1.

8. Expliquez brièvement comment vous pourriez répondre à chaque anomalie.

Règle 4 : on peut la remplacer par $\text{Bins} \rightarrow \text{Bin}$, ou bien remplacer les règles 3 et 4 par $\text{Bins} \rightarrow \text{Bin}^+$.

Règle 3 : on peut la remplacer par $\text{Bins} \rightarrow \text{Bin Bins}$, mais la seconde correction de l'anomalie précédente corrige aussi cette anomalie.

Règle 7 : on peut la remplacer par $\text{Journey} \rightarrow \text{Step Journey}$, ou bien remplacer les règles 6 et 7 par $\text{Journey} \rightarrow \text{Step}^+$. Attention, ici chaque solution est un choix de sémantique. Il faut donc d'abord se demander quelle est la sémantique réellement attendue, et seulement ensuite proposer une syntaxe qui reflète la sémantique.

9. Proposez une variante de cette grammaire qui ne présenterait plus d'anomalies. Ne montrer que les différences avec la grammaire originelle.

- | | |
|--|--|
| 1. $\text{Direction} \rightarrow ('N' \mid 'E' \mid 'S' \mid 'W')$ | 4. $\text{Step} \rightarrow \text{Direction Bins}$ |
| 2. $\text{Bin} \rightarrow ('0' \mid '1')$ | 5. $\text{Journey} \rightarrow \text{Step}^+$ |
| 3. $\text{Bins} \rightarrow \text{Bin}^+$ | |

10. Remplissez le fragment de table de prédiction LL(1) qui suit pour la grammaire de la question 7.

T	N	E	S	W	0	1
Bins	4	4	4	4	3 ou 4	3 ou 4
Journey	6 ou 7	6 ou 7	6 ou 7	6 ou 7	erreur	erreur

Les 2 règles pour Journey doivent s'attendre à lire une direction (N, E, S ou W). Par contre, elles ne s'attendent pas du tout à lire un bit (0 ou 1). La règle 3 de Bins ne peut

être utilisée que pour lire un bit. Par contre, la règle 4 peut s'attendre à lire un bit ou une direction.

Pour de vrai, Bins et Journey doivent s'attendre à lire aussi la fin du texte, mais ce n'était pas dans la question.

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du cours. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Ne pas oublier d'indiquer ses noms et prénoms.

1. Répondez aux questions suivantes relatives au système Xtext en justifiant brièvement la réponse. Bonus pour 3 bonnes réponses.

- **À quoi reconnaît-on les deux vocabulaires sur lesquels sont fondés les grammaires ?**

Le vocabulaire terminal se reconnaît à ce qu'il est introduit par les règles `terminal` (sauf règles `fragment` commentées ci-dessous) ou par une mention directe entre guillemets, ex. `'truc'`. Le vocabulaire non-terminal se reconnaît à ce qu'il est introduit par les règles qui n'ont pas le mot-clé `terminal`.

Vu en TP (ex. dans le sujet du TP).

- **Que signifie la notation `fragment` ?**

La notation `fragment` sert à donner un nom à des constituants du vocabulaire terminal. Cela correspond à la bonne pratique plusieurs fois recommandée en cours et en TD. Attention : ces constituants ne sont pas en eux-mêmes des symboles terminaux.

Vu en TP (ex. dans le sujet du TP).

- **Comment se traduit la double articulation des langages ?**

Un premier niveau d'articulation est réalisé par les règles `terminal` et les règles `fragment`, et pour lesquelles le vocabulaire est celui des caractères. Un second est réalisé par les règles non `terminal`, et lesquelles le vocabulaire est celui qui a été défini par les règles `terminal`.

Attention ! J'ai vu des réponses qui se limitaient à oui ou non. Où est la brève justification qui était demandée ? Ces réponses ne valent rien !

2. Quel effet a la commande « Generate Xtext Artifacts » ?

Cette commande sert à engendrer automatiquement les programmes qui vont exploiter la définition grammaticale du langage. En TP on a vu que cela générerait l'analyseur syntaxique, qui décide si un mot appartient au langage ou non, et un éditeur syntaxique, qui fait la vérification à la volée, colorise les mots clés et affiche les messages d'erreur à la bonne ligne.

Pour de vrai, Xtext engendre d'autres artefacts, mais ce n'était pas le sujet du TP.

Vu en TP (ex. dans le sujet du TP).

3. Rappelez le rôle des séparateurs et comment ils sont réalisés en Xtext.

Les séparateurs sont des éléments du flux d'entrée qui servent de délimitation aux unités lexicales. Ils ne sont pas mentionnés dans la grammaire. En Xtext, ils sont décrits comme des symboles terminaux puis notés comme `hidden`.

Vu en TP (ex. dans le sujet du TP).

4. Quel effet a la commande « Run as Eclipse Application » ?

Cette commande lance un nouvel Eclipse qui embarque les programmes qui ont été générés par Generate Xtext Artifacts, et pour qui le langage décrit par la grammaire est donc devenu natif. C'est comme si Eclipse était lancé avec un nouveau *plugin*.

Vu en TP (ex. dans le sujet du TP)

5. Xtext détecte que votre grammaire est récursive à gauche. Pourquoi proteste-t-il ?

Xtext proteste contre les grammaires récursives à gauche car il produit des analyseurs syntaxiques LL(x). Plus précisément, un analyseur par descente récursive. Ces analyseurs sont allergiques aux grammaires récursives à gauche.

6. Quel dispositif de Xtext permet de résoudre le problème précédent ?

Utiliser une notation d'expression régulière avec une fermeture permet souvent de résoudre ce problème.

7. Donnez un exemple de grammaire récursive à gauche et de la résolution de ce problème en notation Xtext.

$$\text{Expr} : \text{Expr} \text{ 'op' } \text{SousExpr} \quad | \quad \text{SousExpr} ;$$

peut être remplacé par

$$\text{Expr} : \text{SousExpr} (\text{'op'} \text{ SousExpr})^* ;$$

Principe vu en TD et notation vue en TP.

On souhaite étendre le langage WHILE avec une notation d'expression conditionnelle inspirée de la forme (*condition ? alors : sinon*) de Java, mais adapté au style des expressions Lisp (qu'on supposera déjà définies). Par exemple, on écrira

$$(\text{cond } \text{condition } \text{alors } \text{sinon}).$$

On appellera ces expressions des expressions **cond**.

8. Proposez une description en Xtext des expressions cond.

Expr : '(' 'cond' Expr Expr Expr ')' ;

ou bien

Expr : '(' 'cond' cond=Expr alors=Expr sinon=Expr ')' ;

Principe vu en TD et notation vue en TP.

On souhaite étendre la notation d'expression cond introduite plus haut pour prendre en compte une suite de une ou plusieurs conditions à tester. Par exemple, on écrira

(conds condition1 alors1 condition2 alors2 sinon).

On appellera ces expressions des expressions conds.

9. Proposez une description en Xtext des expressions conds.

Expr : '(' 'conds' Conds+ Expr ')' ;
Conds : Expr Expr

ou bien

Expr : '(' 'conds' Conds+ sinon=Expr ')' ;
Conds : cond=Expr alors=Expr

Principe vu en TD et notation vue en TP.

10. Vous devez guider un stagiaire dans l'élaboration de sa première grammaire Xtext. Que lui dites-vous ?

- Rassembler ses souvenirs du cours de TL, imprimer un aide-mémoire pour les notations (*cheat-sheet*), suivre le tutoriel, bien repérer comment les concepts de TL s'alignent avec ceux de Xtext, bien comprendre que Xtext permet de créer des utilitaires nouveaux pour une nouvelle génération d'Eclipse pour qui le langage décrit sera natif.
- Bien repérer toutes les étapes du cycle de vie : écrire une grammaire, en dériver les artefacts, lancer un nouvel Eclipse pour qui le langage décrit est natif, tester la grammaire avec des exemples et des contre-exemples.
- Enfin, ne pas tenter de produire une grammaire d'un coup, et au contraire prévoir un développement graduel de grammaires de plus en plus complexes, à chaque étape duquel on applique tout le cycle de vie rappelé plus haut.

Ce genre de question sert à voir si vous êtes capables d'expliquer dans un environnement professionnel ce que vous faites.